

CONFIDENTIAL · PROJECT REPORT

L-I-M-E.

Testing & Security Research Report

WEEK 1 DELIVERABLES

PREPARED BY

Siboniso Junior

ROLE

**Testing & Security
Lead**

DATE

**Week 1 · Project
Initiation**

Contents

01	Introduction	Framing the mission
02	Part 1 — Security Research	SQL Injection · Hashing · XSS
03	Part 2 — Testing Strategy	7-category checklist
04	Part 3 — Sprint Testing Plan	Sprints 1 – 5
05	Week 1 Summary	Deliverables & next actions
06	Conclusion	Why this matters
07	Questions for the Team	Open decisions

Introduction

"We're building a bridge between students and companies. If that bridge has weak spots, nobody will want to cross it."

Good evening team. As we know for the L-I-M-E project, my responsibility focuses on **Testing and Security**. We need to understand the threats our platform could face and plan how we will keep it safe.

Let me walk you through what I've researched for Week 1 and how it will apply to our project.

Security Research

The three threats that matter most to L-I-M-E right now — and exactly how we stop them.

1 SQL Injection

WHAT IS IT?

Imagine someone filling out a form on our website, but instead of typing their name, they type a command that tricks our database into giving up all its secrets. That's SQL injection. It's one of the oldest and most dangerous web attacks.

WHY DOES L-I-M-E NEED TO WORRY ABOUT THIS?

We're dealing with sensitive data:

- Student login credentials
- Personal student information
- Company registration details
- Project files and descriptions

IF AN ATTACKER GETS IN, THEY COULD...

Steal all user passwords · View private student portfolios
· Access company contact information · Delete or corrupt
our entire database

HOW WE PREVENT IT

1. **Prepared Statements** — Our strongest defense. Instead of directly inserting user input into SQL queries, we use placeholders. The database treats the input as data, not executable code.
2. **Input Validation** — We check every piece of data coming into our system. Student IDs must match a specific format. Email addresses must look like real emails.
3. **Parameterized Queries** — Similar to prepared statements, this separates the query structure from the data being queried.
4. **Least Privilege Principle** — Our database user shouldn't have permission to drop tables or make structural changes. Only enough access to do what's necessary.

WHERE IN L-I-M-E ARE WE MOST VULNERABLE?

- Login and registration forms
- Search functionality (companies searching for students)
- Portfolio upload and management pages
- Anywhere users input text that gets stored

2 Password Hashing

WHAT IS IT?

When a user creates an account, we don't store their actual password. Ever. We store a "hash" — a scrambled version that's mathematically impossible to reverse. Think of it like a fingerprint: unique, but you can't recreate the finger from the print.

WHY DOES L-I-M-E NEED THIS?

Because if our database is ever compromised (we hope it never happens, but we plan for it), we want to protect our users. Students use this password across multiple services. A breach would expose them to identity theft and other attacks.

BEST PRACTICES FOR L-I-M-E

1. Use PHP's `password_hash()` function — it automatically handles the complex security details:

```
// Creating a hash (when user signs up)
$hashedPassword = password_hash($userPassword,
PASSWORD_DEFAULT);
// Storing this in our database

// Verifying a password (when user logs in)
if (password_verify($enteredPassword,
$hashedFromDatabase)) {
```

```
// Password is correct, let them in  
}
```

2. Salt — A random string added to each password before hashing. Even if two students have the same password, their hashes will be different. PHP's `password_hash()` handles this automatically.

3. Slow Hashing Algorithms — We use algorithms like `bcrypt` or `Argon2` that are intentionally slow. This makes it computationally expensive for attackers to crack passwords, but is negligible for legitimate users.

✘ WHAT WE NEVER DO

- Store passwords in plain text
- Use weak hashing algorithms (MD5, SHA1)
- Use the same hash for all users
- Email passwords to users

3 Cross-Site Scripting (XSS)

WHAT IS IT?

This happens when attackers inject malicious scripts into web pages that other users view. Imagine a student writing a portfolio description that contains a script that steals other users' session

cookies. When an employer views that portfolio, their session is compromised.

WHY DOES L-I-M-E NEED TO WORRY ABOUT THIS?

Our platform is all about displaying user-generated content:

- Student portfolio descriptions
- Project titles and descriptions
- Company job postings
- Messages between students and companies
- Comments in the community section

Any of these could be exploited if we're not careful.

TYPES OF XSS WE NEED TO PREVENT

1. **Stored XSS** — Malicious code is permanently stored in our database and served to anyone who views the affected page. This is our biggest risk.
2. **Reflected XSS** — Malicious code is passed through URL parameters or form submissions and immediately reflected back to the user.
3. **DOM-based XSS** — Malicious code manipulates the Document Object Model (the structure of the web page) without involving our server.

PREVENTION STRATEGIES FOR L-I-M-E

1. **Output Escaping** — When we display user-generated content, we escape special characters like `<`, `>`, `&`, `"`, `'`. This turns `<script>` into `<script>`; which browsers display as text, not execute.
2. **Content Security Policy (CSP)** — We tell the browser exactly what sources of scripts are allowed. This prevents execution of unauthorized scripts.
3. **Input Validation** — We validate all input on both the client and server side. This is like having security guards at multiple checkpoints.
4. **Context-Aware Encoding** — HTML, JavaScript, CSS, and URLs all need different escaping rules. What works in one context may fail in another.

KEY AREAS TO FOCUS ON

- Portfolio content display
- Search results page
- Messages between users
- Company opportunity descriptions
- Profile information display

Testing Strategy

A comprehensive testing checklist organized by feature area, with priority levels so we cover everything systematically.

A Authentication & User Management

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Student registration with valid institutional email	HIGH	Account created, verification email sent	Student ID, univ. email
Student registration with invalid email format	HIGH	Rejection with clear error message	Invalid format
Student registration with duplicate email	HIGH	Error: "Account already exists"	Existing email
Company registration with valid company email	HIGH	Account created, pending admin verification	Company email, reg. number

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Company registration with mismatched email domain	HIGH	Rejection: domain doesn't match company	Different domain
Login with correct credentials	HIGH	Successful login, user redirected to dashboard	Valid credentials
Login with incorrect password	HIGH	Error message, no lockout after 3 attempts	Wrong password
Login with non-existent account	HIGH	Generic error (don't reveal if account exists)	Fake credentials
Password reset flow	HIGH	Reset email sent, token works once	Valid email
Session timeout	MEDIUM	User automatically logged out after inactivity	—
Multi-device simultaneous login	MEDIUM	Both sessions active or one invalidated	Two browsers

B Student Portfolio Features

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Upload project with valid file types (PDF, DOCX, ZIP)	HIGH	File uploaded, processed successfully	Sample project files
Upload project with invalid file type (.exe, .sh)	HIGH	Rejection with clear error message	Invalid file types
Upload project exceeding size limit (e.g. 10MB)	HIGH	Rejection: "File too large"	Large file
Add project description with special characters	HIGH	Displayed correctly, properly escaped	Text with <script>, quotes
Delete portfolio item	HIGH	Item removed, confirmation prompt	Existing item
Edit existing portfolio item	HIGH	Changes saved and displayed correctly	—
AI-generated description from uploaded code	MEDIUM	Description generated within reasonable time	Code file
AI skill tagging from uploaded code	MEDIUM	Relevant skills detected and tagged	Code file

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Auto-save draft functionality	LOW	Work preserved without user action	—

C Company Features

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Post internship opportunity	HIGH	Published, visible in student feed	Complete job details
Post workshop opportunity	HIGH	Published, visible to students	Workshop details
Search for students by skill	HIGH	Relevant students displayed	Skill keywords
Search for students by project type	HIGH	Relevant students displayed	Project categories
View student portfolio	HIGH	Full profile displayed correctly	Student profile
Filter students by multiple criteria	MEDIUM	Accurate filtering results	Multiple filters
Browse student recommendations	MEDIUM	AI-suggested students match criteria	—

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Company receives notification on application	HIGH	Real-time notification delivered	—
Internal messaging system	HIGH	Messages sent and received successfully	—

D AI Feature Testing

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Generate portfolio description from assignment	HIGH	Coherent description returned	Assignment text
Generate project improvement suggestions	MEDIUM	Relevant, actionable suggestions	Project details
Auto-tag skills from uploaded code	HIGH	Accurate skill tags detected	Code in multiple languages
Personalized internship recommendations	HIGH	Relevant opportunities suggested	Student skills and activity
Response time for AI calls	MEDIUM	Under 3 seconds	API calls

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
AI fallback when API unavailable	MEDIUM	Graceful error, user informed	—
Rate limiting on AI features	LOW	Prevent abuse of API	Rapid repeated requests

E Integration Testing

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
LinkedIn profile integration	MEDIUM	Profile data displayed correctly	LinkedIn URL
GitHub repository integration	MEDIUM	Repositories displayed	GitHub URL
Link verification (external platforms)	MEDIUM	Validation of public profile	Invalid URLs
Student ID verification with institutional system	HIGH	Successful verification	Valid/invalid student IDs
Email domain verification for companies	HIGH	Domain validation works	—
Email verification flow	HIGH	User can't access account until verified	—

F Security Testing

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
SQL Injection attempt in login	HIGH	Query fails, user not logged in	SQL injection payloads
SQL Injection attempt in search	HIGH	Query sanitized, returns no/correct results	SQL injection payloads
XSS attempt in profile description	HIGH	Script rendered as text, not executed	<script>alert()</script>
XSS attempt in message content	HIGH	Script rendered as text, not executed	<script>alert()</script>
CSRF attempt on forms	HIGH	Requests without token rejected	Cross-site request
Password complexity enforcement	HIGH	Weak passwords rejected	Weak passwords
HTTPS / SSL encryption	HIGH	All traffic encrypted	—
File upload vulnerability (ZIP slip)	MEDIUM	Malicious uploads rejected	Malicious zip files

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Admin access controls	HIGH	Non-admin can't access admin areas	—
Session fixation attempt	HIGH	Session regenerated after login	Session hijacking attempt

G Usability & Accessibility Testing

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Platform works on major browsers (Chrome, Firefox, Safari)	HIGH	Same functionality across browsers	—
Mobile responsive design	HIGH	Usable on phone and tablet	—
Screen reader compatibility	LOW	Basic navigation possible	—
Clear error messages	HIGH	User understands what went wrong	—
Loading time of dashboard	HIGH	Under 3 seconds	—

TEST CASE	PRIORITY	EXPECTED RESULT	DATA NEEDED
Navigation menu consistency	HIGH	Same menu items across pages	—

Sprint Testing Plan

Our Agile approach means testing happens throughout the project, not just at the end. Here's what we'll test in each sprint.

Focus: User authentication, database setup, and basic UI

WHAT TO TEST

- ☒ Database schema — can we connect and query correctly?
- ☒ Login system — does authentication work securely?
- ☒ Registration flow — can users sign up properly?
- ☒ Password hashing — are passwords stored securely?
- ☒ Email verification — does the verification process work?
- ☒ Security basics — SQL injection prevention in login/registration
- ☒ Session management — do sessions start and end correctly?

TEST DATA NEEDED

- 5 test students with real and fake data
- 3 test companies with legitimate details
- SQL injection test payloads
- XSS test scripts

Focus: Student and company dashboards, portfolio creation

WHAT TO TEST

- ☒ Student dashboard loads correctly
- ☒ Company dashboard loads correctly
- ☒ Upload portfolio projects (multiple file types)
- ☒ Display portfolio content correctly
- ☒ Edit/delete portfolio items
- ☒ User-specific content (students see student view, companies see company view)
- ☒ Search functionality in the student view
- ☒ File upload security (prevent malicious files)
- ☒ XSS prevention in displayed portfolio content

TEST DATA NEEDED

- Various file types (PDF, DOCX, ZIP, images, code files)
- Large files (near the size limit)
- Files with potential security risks

Sprint 3 · Core Company Features

Weeks 5–6

Focus: Company features — posting opportunities, searching

WHAT TO TEST

- ☒ Post internship opportunities
- ☒ Post workshop opportunities
- ☒ Search for students by skills
- ☒ View student profiles
- ☒ Company verification process
- ☒ Admin approval of company accounts
- ☒ Student applications to opportunities
- ☒ Notification system basics
- ☒ SQL injection in search functionality

TEST DATA NEEDED

- Sample job postings
- Diverse student profiles for search testing
- Company verification documents

Focus: AI features, recommendations, messaging

WHAT TO TEST

- ☒ AI portfolio description generation
- ☒ AI skill tagging from code
- ☒ Internship recommendations
- ☒ Messaging between students and companies
- ☒ Internal communication system
- ☒ AI feature response times
- ☒ AI fallback mechanisms
- ☒ API rate limiting and error handling
- ☒ Integration with LinkedIn and GitHub (if included)

TEST DATA NEEDED

- Code files in multiple languages (Python, JavaScript, Java)
- Student activity data for recommendations
- API keys for testing

Sprint 5 · Testing, Optimization & Deployment

Weeks 9–
10

Focus: End-to-end testing, performance, and launch

WHAT TO TEST

- ☒ Complete user journey (registration → portfolio → application → hiring)
- ☒ Cross-browser compatibility
- ☒ Mobile responsiveness
- ☒ Load testing (simulate multiple users)
- ☒ Comprehensive security audit
- ☒ Data integrity after operations
- ☒ Backup and recovery procedures
- ☒ Production deployment validation
- ☒ All integration points working

TEST DATA NEEDED

- Full set of real-looking user data
- Multiple simultaneous users for load testing
- Edge cases and boundary conditions

Summary: My Week 1

For this first week, I've completed:

DELIVERABLE	STATUS	DETAILS
SQL Injection Research	COMPLETE	Understood, documented prevention methods
Password Hashing Research	COMPLETE	Using PHP password_hash() as our approach
XSS Research	COMPLETE	Output escaping is our primary defense
Testing Checklist	COMPLETE	7 categories covering all platform features
Sprint Testing Plan	COMPLETE	Tests mapped across 5 sprints
Security Recommendations	COMPLETE	Practical, implementable measures

What needs to happen in Week 1/2

1. SET UP SECURITY MEASURES

- Configure database with prepared statement support
- Implement password hashing on registration/login

- Add basic input validation to all forms
- Set up error logging (not revealing to users)

2. CREATE TEST ENVIRONMENT

- Set up a separate testing database
- Create test user accounts (students + companies)
- Prepare test files for portfolio uploads

3. START WITH FOUNDATIONAL TESTING

- Test registration flow
- Test login security (try SQL injection)
- Verify password hashing works

4. DOCUMENT EVERYTHING

- Security decisions made
- Testing results from each sprint
- Any vulnerabilities found and fixed

Conclusion

Security and testing aren't just boxes we check — they're what makes L-I-M-E trustworthy. Students are trusting us with their academic work and personal information. Companies are trusting

us with their hiring processes. We owe it to both to build something secure.

The research I've done this week gives us a solid foundation. SQL injection, password hashing, and XSS are our biggest web security threats, and we now have clear strategies to prevent them. Our testing checklist covers every feature we're building, and our sprint plan ensures we're testing throughout development, not leaving it to the end.

When I present this to the team, I'll emphasize that these aren't just technical requirements — they're the difference between a platform users trust and one they avoid.

A secure L-I-M-E is a successful L-I-M-E.

Questions for the Team

- 1 Do we want to implement two-factor authentication for sensitive actions?

2 What is our policy on password reset method (email vs. security questions)?

3 How long should we keep the user data after account deletion?

PREPARED BY

Siboniso Junior

Testing & Security Lead · L-I-M-E Platform

Week 1 — Project Initiation